

Kapitel 2 · Fertige Bausteine nutzen

Fahrradverleih · Voraussetzung: Kapitel 1

Das lernen Sie

Sie nutzen vorhandene Funktionen und Methoden, ersetzen ein Dezimalkomma und erzeugen aus Eingaben ein Datum. Zwei alternative Wege zur Datumseingabe ergänzen die Vorlage.

1. Library, Modul und Import

Allgemeine Grundlagen: Eine Bibliothek enthält wiederverwendbare Bausteine. Ein Modul ist eine importierbare Einheit; ein Paket kann mehrere Module bündeln. Durch Importieren werden die Bausteine dem laufenden Programm zugänglich gemacht.

Merksatz – Import: Erst importieren, dann den importierten Namen verwenden.

import datetime führt zum Aufruf **datetime.date(...)**;

from datetime import date ermöglicht **date(...)**.

```
from datetime import date
termin = date(2026, 10, 5)
print(termin) # Ausgabe: 2026-10-05
```

Allgemeine Grundlagen: Funktionen werden mit Argumenten aufgerufen und können einen Rückgabewert liefern. Methoden werden über einen Punkt an einem Objekt oder einer Klasse aufgerufen. **date** und **datetime** sind Klassen; ihre Verwendung wird hier auf die benötigten Aufrufe beschränkt.

Merksatz – Funktion und Methode: `len(text)` ist ein Funktionsaufruf, `text.replace(...)` ein Methodenaufruf. Das Ergebnis muss verwendet oder gespeichert werden, damit es für den weiteren Ablauf verfügbar ist.

```
text = "A-B"
print(len(text)) # Ausgabe: 3
print(text.replace("-", "/")) # Ausgabe: A/B
```

Anwendung: Fahrradverleih

Eine Library (Bibliothek) stellt fertige Programmbausteine bereit. Ein Modul bündelt solche Bausteine unter einem Namen. Python bringt eine Standardbibliothek mit; dazu gehört **datetime** für Datum und Uhrzeit. Andere Bibliotheken, etwa der MySQL-Treiber in Kapitel 4, müssen zusätzlich installiert werden.

```
from datetime import date, datetime
```

Mit **from ... import ...** holen Sie die angegebenen Namen aus dem Modul. **date** steht für reine Datumswerte, **datetime** für Datum mit Uhrzeit. Für diese Reihe genügt es, beide wie fertige Werkzeuge zu verwenden; eigene Klassen oder Funktionen brauchen Sie nicht zu definieren.

Eine Funktion rufen Sie mit **name(argument)** auf, etwa **len(rahmenNr)**. Eine Methode gehört zu einem Wert oder einem anderen Baustein und wird über einen Punkt aufgerufen, etwa **text.replace(",", ".")**. Argumente liefern die benötigten Angaben; der Rückgabewert ist das Ergebnis des Aufrufs.

2. replace() und len()

Allgemeine Grundlagen: Zeichenketten sind unveränderlich. `replace(alt, neu)` erzeugt einen neuen Text; ohne Mengenbegrenzung werden alle passenden Stellen ersetzt. `len()` liefert bei einem Text die Zeichenanzahl als `int`.

Merksatz – `replace()`: Der ursprüngliche Text bleibt bestehen, bis Sie einer Variablen das neue Ergebnis zuweisen.

```
text = "rot-rot"
neu = text.replace("rot", "blau")
print(text) # Ausgabe: rot-rot
print(neu)  # Ausgabe: blau-blau
```

Merksatz – `len()`: Auch Leerzeichen und Trennzeichen zählen mit; der leere Text hat die Länge null.

```
print(len("A B")) # Ausgabe: 3
print(len(""))   # Ausgabe: 0
```

Anwendung: Fahrradverleih

```
preis_text = input("Tagesmietpreis: ")
preis_text = preis_text.replace(",", ".")
tagesmietpreis = float(preis_text)
```

`replace()` liefert hier einen neuen Text, in dem jedes Komma durch einen Punkt ersetzt ist. Deshalb speichern wir das Ergebnis wieder in `preis_text`. Aus "12,50" wird "12.50"; erst `float()` macht daraus eine Zahl. Auch "12.50" bleibt verwendbar.

```
rahmenNr = input("Rahmennummer: ")
print("Zeichenanzahl:", len(rahmenNr))
```

`len()` liefert die Anzahl der Zeichen, einschließlich Bindestrichen und Leerzeichen. Es prüft noch keine Höchstlänge; dafür kommt in Kapitel 3 eine Bedingung hinzu.

Kapitel 2 · Drei Wege zum Datum

3. date(): Jahr, Monat und Tag zusammensetzen

Allgemeine Grundlagen: Ein Datumswert ist kein bloßer Text. `date` speichert Jahr, Monat und Tag ohne Uhrzeit. Die drei Argumente müssen ganze Zahlen sein und zusammen einen gültigen Kalendertag ergeben. Das gilt auch für Schaltjahre.

Merksatz – `date()`: Die Reihenfolge lautet immer Jahr, Monat, Tag. Die Textdarstellung bei `print()` lautet Jahr-Monat-Tag.

```
from datetime import date
termin = date(2028, 2, 29)
print(termin) # Ausgabe: 2028-02-29
```

Anwendung: Fahrradverleih

```
jahr = int(input("Jahr: "))
monat = int(input("Monat: "))
tag = int(input("Tag: "))
anschaffungsdatum = date(jahr, monat, tag)
```

`date()` erzeugt einen Datumswert aus drei ganzen Zahlen in der Reihenfolge Jahr, Monat, Tag. `print(anschaffungsdatum)` zeigt beispielsweise 2026-09-08. Genau diesen Weg verwendet `db_kom_check.py`. Ein Datum wie der 31. Februar ist ungültig und führt zu `ValueError`.

4. split(): Einen Datumstext zerlegen (Optional)

Allgemeine Grundlagen: `split(trennzeichen)` zerlegt Text und liefert eine Liste von Teiltextren. Für diese Aufgabe genügt es, die Teile direkt auf mehrere Variablen zu verteilen. Dabei müssen die Anzahl der Variablen und die Anzahl der Teile übereinstimmen.

Merksatz – `split()`: Aus Text entstehen Teiltexpte.

```
angabe = "05/10/2026"
tag_text, monat_text, jahr_text = angabe.split("/")
print(tag_text, monat_text, jahr_text) # Ausgabe: 05 10 2026
tag = int(tag_text) # Erst jetzt entsteht die ganze Zahl 5.
```

Beispiel zur Grenze: "05//2026" liefert bei `split("/")` drei Teile, aber der mittlere Teil ist leer. Eine passende Anzahl von Teilen garantiert daher noch kein gültiges Datum.

Anwendung: Fahrradverleih (Optional)

```
datum_text = "08.09.2026"
tag_text, monat_text, jahr_text = datum_text.split(".")
datum_split = date(int(jahr_text), int(monat_text), int(tag_text))
```

`split(".")` trennt den Text an den Punkten. Die drei Teiltexpte werden hier auf drei Variablen verteilt. Diese Mehrfachzuweisung benötigt genau drei Teile; zwei oder vier Teile führen zu einem Fehler. Die Teile sind weiterhin Text und werden vor `date()` mit `int()` umgewandelt.

5. strptime(): Einen Datumstext nach Muster lesen (Optional)

Allgemeine Grundlagen: Ein Formatmuster beschreibt die Bedeutung und Reihenfolge der Bestandteile eines Datumstextes. Die Trennzeichen im Muster müssen zur Eingabe passen. `datetime.strptime()` erzeugt einen `datetime`-Wert; bei einem reinen Datums muster wird die Uhrzeit auf Mitternacht gesetzt.

Merksatz – `strptime()`: Das Muster erklärt den Text: %d ist der Tag, %m der Monat, %Y das vierstellige Jahr. `.date()` entnimmt anschließend das reine Datum.

```
from datetime import datetime
zeitpunkt = datetime.strptime("05/10/2026", "%d/%m/%Y")
print(zeitpunkt.date()) # Ausgabe: 2026-10-05
```

Beispiel zur Grenze: Der Text "05/10/2026" passt nicht zum Muster "%d.%m.%Y", weil Schrägstriche statt Punkte verwendet werden.

Anwendung: Fahrradverleih (Optional)

```
datum_zeit = datetime.strptime(datum_text, "%d.%m.%Y")
datum_format = datum_zeit.date()
```

`strptime()` liest den Text nach einem Formatmuster: %d steht für den Tag, %m für den Monat und %Y für das vierstellige Jahr. Die Punkte passen zu den Trennzeichen der Eingabe. Das Ergebnis ist ein `datetime`-Wert. Dessen Methode `.date()` liefert den reinen Datumswert für die Datenbank.

Unterscheiden Sie `date(jahr, monat, tag)`, das ein Datum erzeugt, von `datum_zeit.date()`, das den Datumsteil entnimmt. `strptime()` meldet bei unpassendem Format oder ungültigem Datum einen `ValueError`.

Welche Variante brauche ich?

Merksatz – Auswahl: Drei bereits vorhandene Zahlen passen zu `date()`; getrennte Teilttexte entstehen mit `split()`; ein Datumstext mit bekanntem Muster passt zu `strptime()`.

Allgemeines Beispiel: Für die drei Zahlen 2026, 10 und 5 genügt `date(2026, 10, 5)`. Für "05.10.2026" kann `datetime.strptime(text, "%d.%m.%Y").date()` verwendet werden. Ein Programm muss diese Wege nicht alle gleichzeitig einsetzen.

Die Vorlage nutzt nur `date(jahr, monat, tag)`. `split()` und `strptime()` sind Alternativen, falls das Datum als eine einzige Eingabe erfolgen soll. In der nächsten Übung vergleichen Sie die drei Ergebnisse; im Abschlussprogramm genügt die Variante der Vorlage.

Kapitel 2 · Pflichtübung

Auftrag: Eingaben aufbereiten und Datumswege vergleichen

Erstellen Sie `uebung_02.py`. Nutzen Sie die Grundlagen aus Kapitel 1 und die neuen Bausteine; verwenden Sie noch keine `if`-Abfrage, Schleife oder Datenbank.

1. Importieren Sie `date` und `datetime` aus dem Modul `datetime`.
2. Fragen Sie eine Rahmennummer ab und geben Sie mit `len()` deren Zeichenanzahl aus.
3. Fragen Sie den Tagesmietpreis ab. Ersetzen Sie mit `replace()` ein Komma durch einen Punkt und wandeln Sie das Ergebnis in `float` um.
4. Fragen Sie Jahr, Monat und Tag einzeln ab und erzeugen Sie mit `date()` das Anschaffungsdatum.

Optional:

5. Fragen Sie dasselbe Datum zusätzlich als Text im Format `TT.MM.JJJJ` ab. Zerlegen Sie es mit `split()`, verteilen Sie die Teile auf drei Variablen und erzeugen Sie nach der Umwandlung mit `int()` einen zweiten Datumswert.
6. Verarbeiten Sie denselben Text zusätzlich mit `datetime.strptime()` und `.date()`. Geben Sie die drei Datumswerte sowie den Preis beschriftet aus.

Probelauf und erwartete Werte

Eingaben in Reihenfolge: AB1234, 12,50, 2026, 9, 8, 08.09.2026.

Zeichenanzahl: 6
Tagesmietpreis: 12.5 Euro
Datum aus Einzelwerten: 2026-09-08

Optional

Datum mit `split`: 2026-09-08
Datum mit `strptime`: 2026-09-08

Wiederholen Sie den Test mit 12.50 als Preis. Das Ergebnis soll gleich bleiben. Nutzen Sie für die Datumstests echte Kalendertage und halten Sie das geforderte Format ein.

Erledigt, wenn ...

Alle Ausgaben stimmen und Sie können erklären, warum `replace()` vor `float()` ausgeführt wird, warum `split()` allein noch kein Datum liefert und wozu `.date()` nach `strptime()` dient.

Die Musterlösung steht in `loesung.py`. Die dreifache Datumserzeugung dient nur dem Vergleich und ist im späteren Datenbankprogramm nicht erforderlich.

Zum Nachlesen

Datum und `strptime()`: <https://docs.python.org/3/library/datetime.html>

Textmethoden: <https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str>

`len()`: <https://docs.python.org/3/library/functions.html#len>